LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 9
Student Name: Gorapalli Sravya
Roll No.: A24126552257
Date: 30-10-2026

---

1. TITLE

Develop a Full-Stack Web Application Using Express.js, ReactJS and MongoDB

2. AIM

To develop a full-stack Student Management web application in which a ReactJS front end communicates with an Express.js REST API that stores its data in a MongoDB database.

3. OBJECTIVE

The objectives of this experiment are:

- To build a REST API with Express.js that supports create, read, update and delete operations.
- To connect the API to MongoDB (collegeDB.students) using Mongoose and define a schema with validation.
- To build a ReactJS front end that uses state, effects and the fetch API to talk to the back end.
- To serve the built React application from the Express server so the whole application runs on one port.
- To verify the complete data flow from the browser to the database and back.

4. THEORY

A full-stack application has a front end that the user sees, a back end that holds the business logic, and a database that stores the data. In this experiment the front end is built with ReactJS, the back end with Node.js and Express.js, and the database is MongoDB, a document database that stores records as JSON-like documents.

Express.js is a web framework for Node.js that defines routes such as GET /api/students and POST /api/students. Mongoose is an object-document mapper that lets the server describe a Student schema, apply validation rules (required fields, a marks range of 0 to 100) and perform database operations through methods such as find(), create(), findByIdAndUpdate() and findByIdAndDelete().

ReactJS builds the user interface from components. The useState hook keeps the list of students, the form values and error messages, while the useEffect hook loads the students when the page opens. When the user adds, edits or deletes a student, React sends an HTTP request with fetch(), Express updates MongoDB, and React reloads the list so the screen always matches the database.

The request flow is: React component -> fetch() request -> Express route -> Mongoose -> MongoDB -> JSON response -> React state -> re-rendered page. The React project is compiled with Vite into static files, and Express serves those files, so the application is opened at a single address.

5. ALGORITHM / PROCEDURE

1. Create a project folder and initialize it using npm init -y.
2. Install the back-end packages using npm install express mongoose cors.
3. Create models/Student.js defining the schema (rollNo, name, branch, marks) with validation.
4. Create server.js with Express routes for GET, POST, PUT and DELETE on /api/students.
5. Connect to MongoDB (collegeDB) using mongoose.connect() and start the server after the connection succeeds.
6. Create the React project in the client folder using Vite and install its dependencies.
7. Create App.jsx with state for students, the form, the error message and the row being edited.
8. Use useEffect to load students from the API when the page opens.
9. Write functions to add a student (POST), edit marks (PUT) and delete a student (DELETE) using fetch().
10. Build the form, the error message area and the students table in the component and style them with App.css.
11. Build the React application using npm run build so that the client/dist folder is created.
12. Serve client/dist from Express using express.static().
13. Start the server using node server.js and open http://localhost:5000 in the browser.
14. Test adding, reloading, editing, deleting and invalid input, and verify the stored documents in the database.

6. PROGRAM

Terminal commands

```
npm init -y
npm install express mongoose cors
cd client
npm install
npm run build
cd ..
node server.js
```

models/Student.js

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema(
  {
    rollNo: { type: String, required: [true, "Roll No is required"], unique:
true, trim: true },
    name: { type: String, required: [true, "Name is required"], trim: true },
    branch: { type: String, default: "CSE-AIML" },
    marks: {
      type: Number,
      min: [0, "Marks must be between 0 and 100"],
      max: [100, "Marks must be between 0 and 100"],
      default: 0,
    },
```

```
  },
  { timestamps: true }
);

module.exports = mongoose.model("Student", studentSchema);
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const path = require("path");
const Student = require("./models/Student");

const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, "client", "dist")));

const errorMessage = (err) =>
  Object.values(err.errors || {}).map((e) => e.message).join(", ") || err.message;

// READ - all students
app.get("/api/students", async (req, res) => {
  const students = await Student.find().sort({ createdAt: 1 });
  res.json(students);
});

// CREATE - add a student
app.post("/api/students", async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ message: errorMessage(err) });
  }
});

// UPDATE - modify a student
app.put("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ message: errorMessage(err) });
  }
});

// DELETE - remove a student
app.delete("/api/students/:id", async (req, res) => {
  const student = await Student.findByIdAndDelete(req.params.id);
  if (!student) return res.status(404).json({ message: "Student not found" });
  res.json({ message: "Student deleted" });
});

const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/collegeDB";
const PORT = process.env.PORT || 5000;
```

```
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log("MongoDB connected: collegeDB");
    app.listen(PORT, () => console.log(`Server running at http://localhost:$
{PORT}`));
  })
  .catch((err) => console.error("MongoDB connection error:", err.message));
```

client/src/App.jsx

```
import { useEffect, useState } from "react";

const API = "/api/students";
const BRANCHES = ["CSE-AIML", "CSE", "ECE", "MECH"];
const emptyForm = { rollNo: "", name: "", branch: "CSE-AIML", marks: "" };

export default function App() {
  const [students, setStudents] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [error, setError] = useState("");
  const [editId, setEditId] = useState(null);
  const [editMarks, setEditMarks] = useState("");

  const loadStudents = async () => {
    const res = await fetch(API);
    setStudents(await res.json());
  };

  useEffect(() => {
    loadStudents();
  }, []);

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const addStudent = async (e) => {
    e.preventDefault();
    const body = { ...form, marks: form.marks === "" ? undefined :
Number(form.marks) };
    const res = await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    const data = await res.json();
    if (!res.ok) {
      setError(data.message);
      return;
    }
    setError("");
    setForm(emptyForm);
    loadStudents();
  };

  const saveMarks = async (id) => {
    const res = await fetch(`${API}/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ marks: Number(editMarks) }),
    });
    const data = await res.json();
    if (!res.ok) {
      setError(data.message);
      return;
    }
    setError("");
```

```
    setEditId(null);
    loadStudents();
  };

  const deleteStudent = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    loadStudents();
  };

  return (
    <div className="container">
      <h1>Student Management System</h1>
      <p className="sub">React + Express.js + MongoDB (collegeDB.students)</p>

      <form className="card form" onSubmit={addStudent}>
        <input name="rollNo" placeholder="Roll No" value={form.rollNo}
onChange={handleChange} />
        <input name="name" placeholder="Name" value={form.name}
onChange={handleChange} />
        <select name="branch" value={form.branch} onChange={handleChange}>
          {BRANCHES.map((b) => (
            <option key={b}>{b}</option>
          ))}
        </select>
        <input name="marks" type="number" placeholder="Marks" value={form.marks}
onChange={handleChange} />
        <button type="submit">Add Student</button>
      </form>

      {error && <div className="error">{error}</div>}

      <div className="card">
        <h2>Students ({students.length})</h2>
        {students.length === 0 ? (
          <p className="empty">No students found. Add one using the form above.</p>
        ) : (
          <table>
            <thead>
              <tr>
                <th>Roll No</th>
                <th>Name</th>
                <th>Branch</th>
                <th>Marks</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody>
              {students.map((s) => (
                <tr key={s._id}>
                  <td>{s.rollNo}</td>
                  <td>{s.name}</td>
                  <td>{s.branch}</td>
                  <td>
                    {editId === s._id ? (
                      <input
                        className="edit-input"
                        type="number"
                        value={editMarks}
                        onChange={(e) => setEditMarks(e.target.value)}
                      />
                    ) : (
                      s.marks
                    )}
                  </td>
                  <td>
                    {editId === s._id ? (
                      <>
```

```
                        <button className="save" onClick={() =>
saveMarks(s._id)}>Save</button>
                        <button className="cancel" onClick={() =>
setEditId(null)}>Cancel</button>
                      </>
                    ) : (
                      <>
                        <button className="edit" onClick={() => { setEditId(s._id);
setEditMarks(s.marks); }}>Edit</button>
                        <button className="delete" onClick={() =>
deleteStudent(s._id)}>Delete</button>
                      </>
                    )}
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
      </div>
    </div>
  );
}
```

7. CODE EXPLANATION

Tag / Code	Explanation
express.json()	Middleware that parses JSON request bodies
express.static("client/dist")	Serves the built React application from the Express server
new mongoose.Schema({...})	Defines the student fields and validation rules (required, min, max)
Student.find() / create()	Reads all students / inserts a new student document
findByIdAndUpdate() / findByIdAndDelete()	Updates or removes a student using its _id
res.status(201 / 400 / 404)	Returns Created, validation error or Not Found responses
mongoose.connect()	Connects to the collegeDB database before the server starts listening
useState()	Stores students, form values, the error message and the row being edited
useEffect(() => {...}, [])	Loads the students once when the page first opens
fetch(API, { method, body })	Sends POST, PUT and DELETE requests to the Express API
loadStudents()	Reloads the list after every change so the screen matches the database

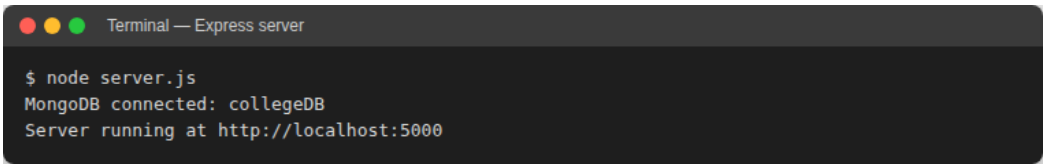
8. TEST CASES AND EXECUTION DATA

The Express server and the React application were run exactly as shown in the Program section and tested in a browser. The official mongod binary could not be downloaded in this sandboxed environment, so the database for these runs was FerretDB v1.24, an open-source database server that speaks the MongoDB protocol. It was used through the same Mongoose code and the same connection string (mongodb://127.0.0.1:27017/collegeDB); on a machine with MongoDB installed the code runs unchanged.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Start the back-end server	Connects to MongoDB and starts listening	MongoDB connected; server running on port 5000	Pass
TC-02	Open the React app with an empty collection	A 'No students found' message is shown	Empty-state message displayed	Pass
TC-03	Add three students using the form	All three students appear in the table	3 students listed, heading shows (3)	Pass
TC-04	Check the database after adding	3 documents stored in collegeDB.students	3 documents found, each with an _id	Pass
TC-05	Reload the page	The students are still displayed	All 3 students loaded again from the database	Pass
TC-06	Edit a student's marks (68 to 74)	The table shows the new marks	Marks updated to 74	Pass
TC-07	Delete a student	The student is removed from the table	Student removed; 2 remain	Pass
TC-08	Add a student with the Name left empty	An error message is shown and nothing is saved	'Name is required' displayed; no record saved	Pass

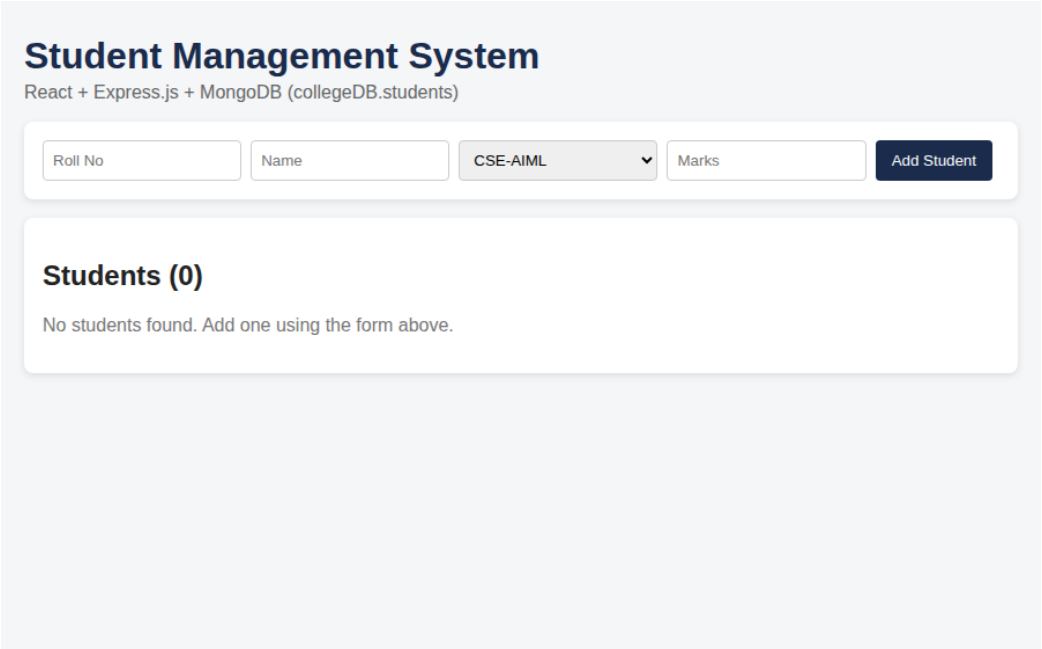
Test Case Output

TC-01 — *Connects to MongoDB and starts listening*



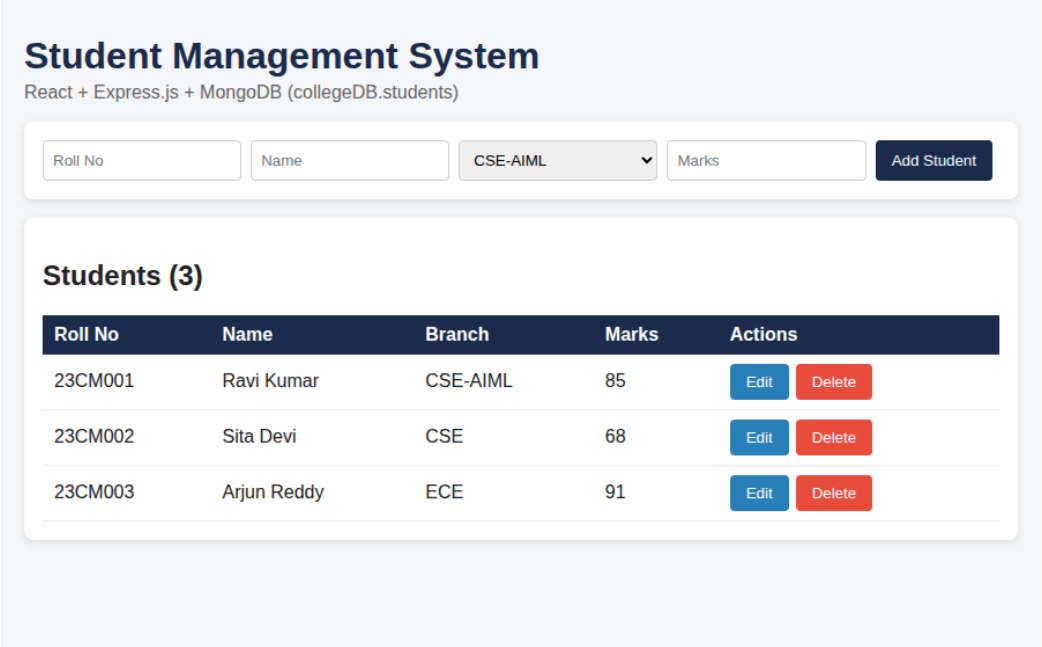
Server log: MongoDB connected and Express listening on port 5000

TC-02 — *A 'No students found' message is shown*



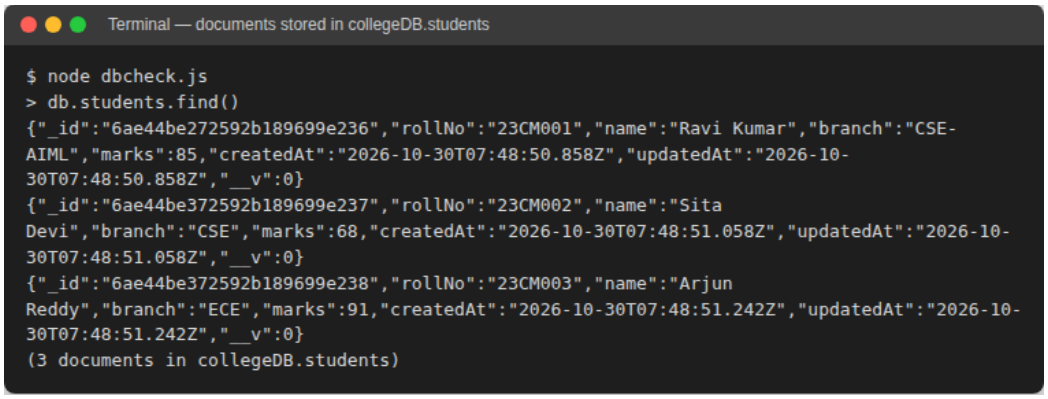
React app opened with an empty students collection

TC-03 — *All three students appear in the table*



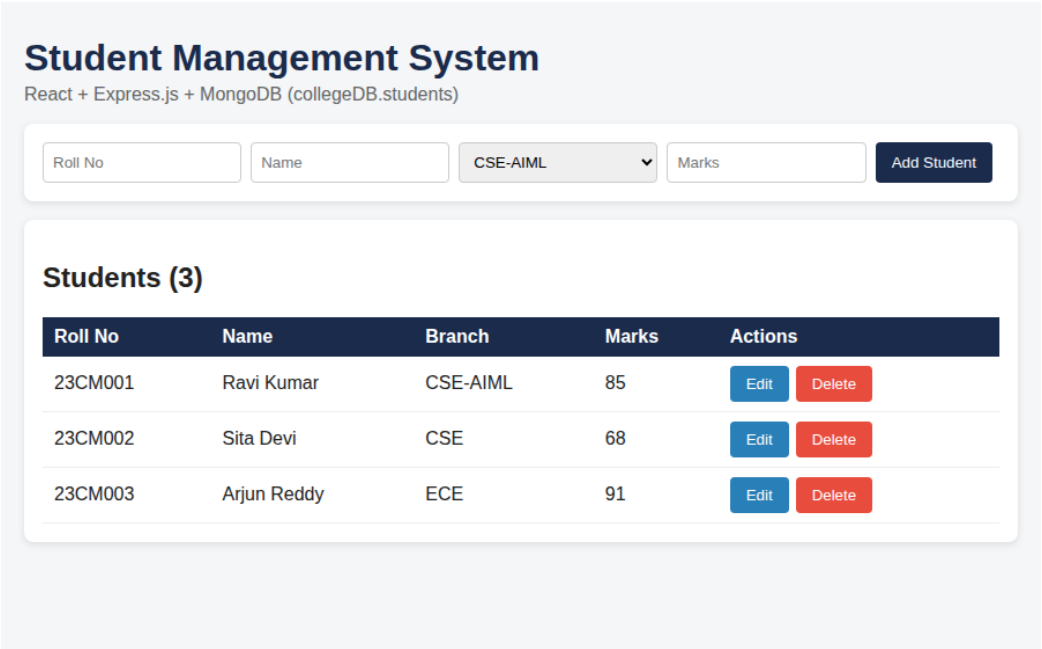
Three students added through the form and listed in the table

TC-04 — *3 documents stored in collegeDB.students*



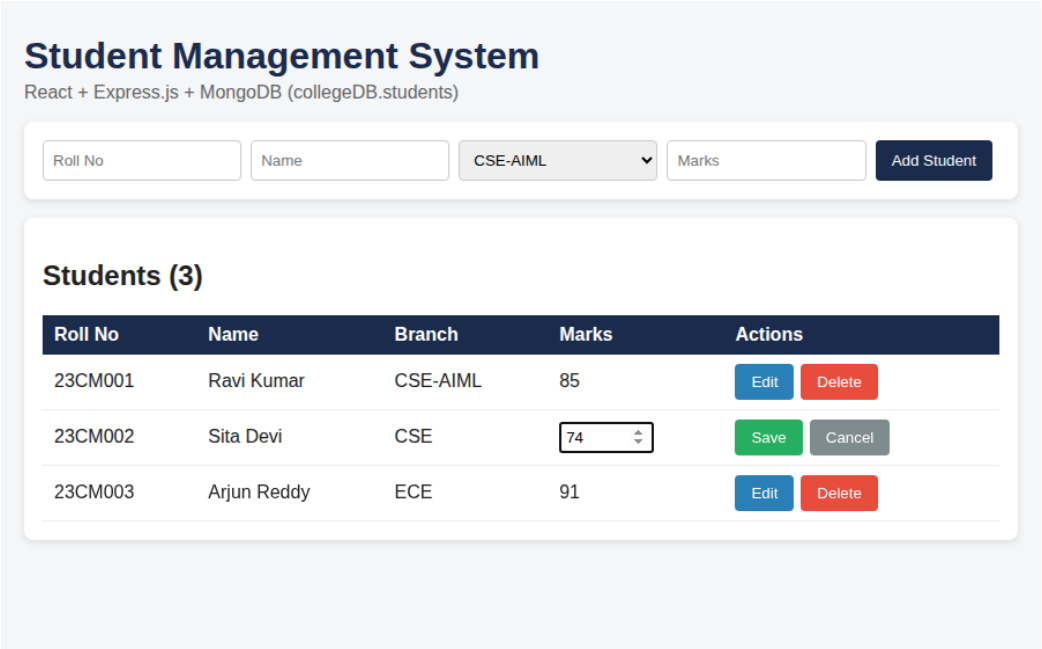
The same three students read directly from the collection, each with its own _id

TC-05 — *The students are still displayed*



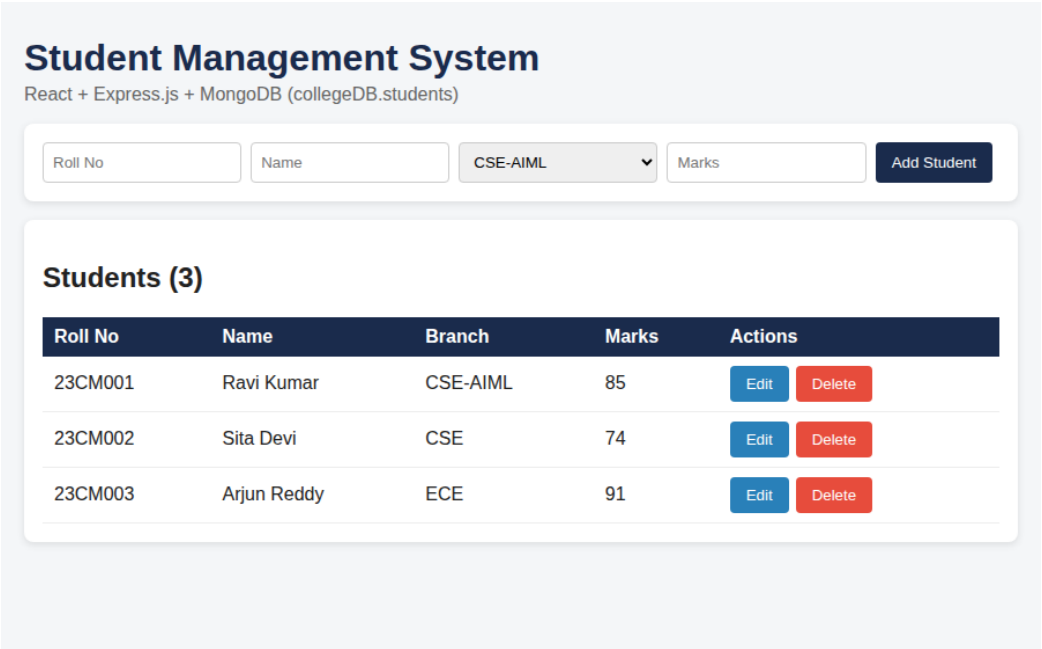
After reloading the page, the data is loaded again from MongoDB

TC-06 — *Edit mode opens an input for marks*



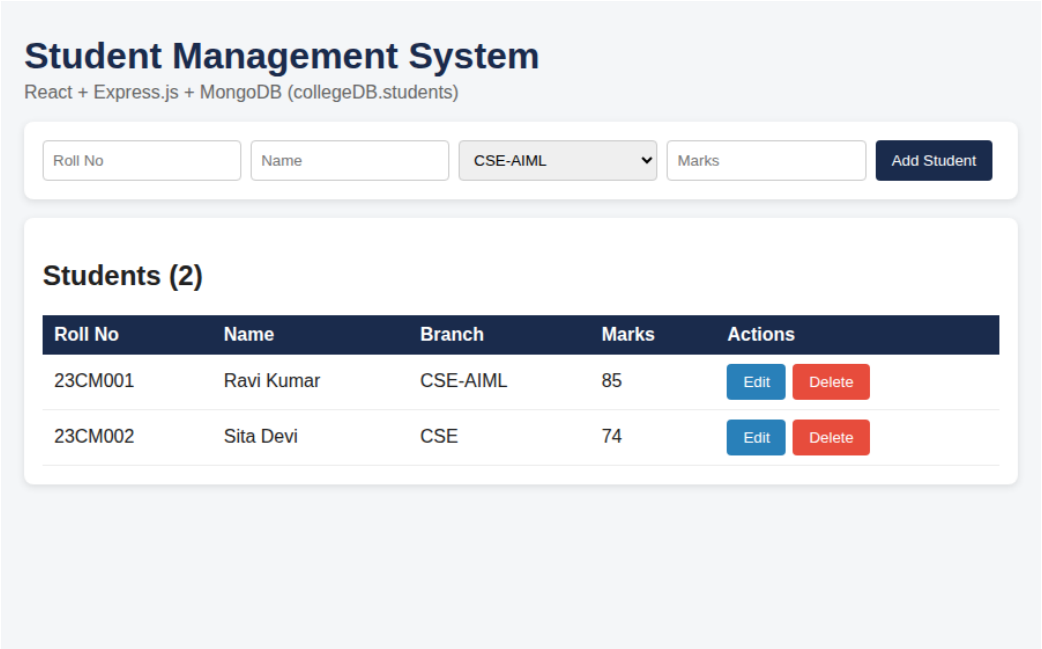
Edit clicked on Sita Devi; the marks cell becomes an input holding the new value

TC-06 — *The table shows the new marks*



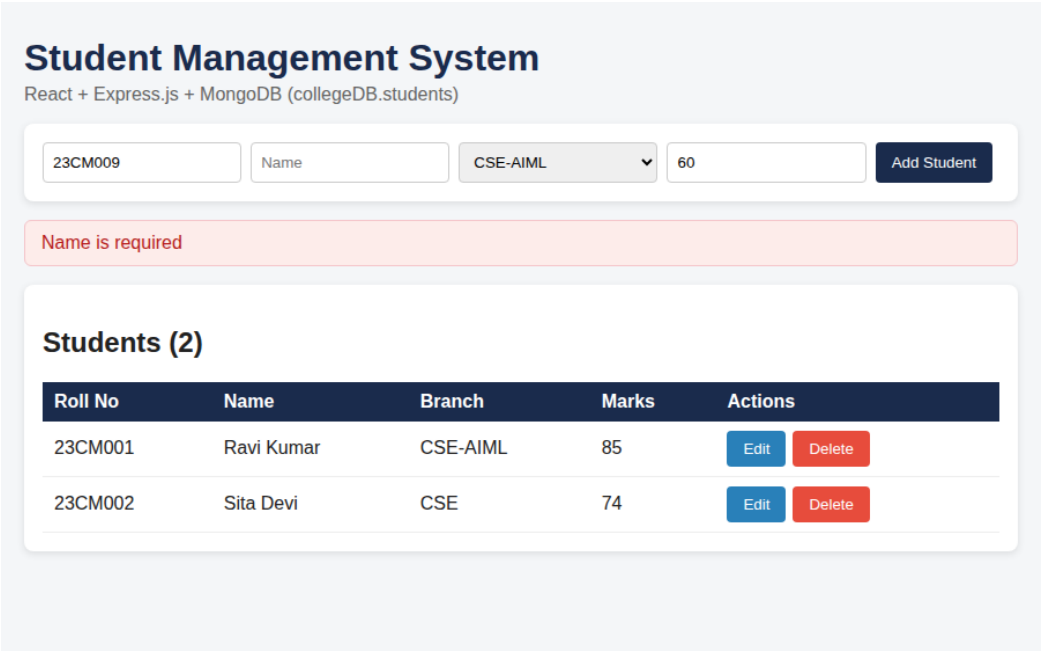
After Save, the marks for Sita Devi change from 68 to 74

TC-07 — *The student is removed from the table*



Arjun Reddy deleted; the heading now shows 2 students

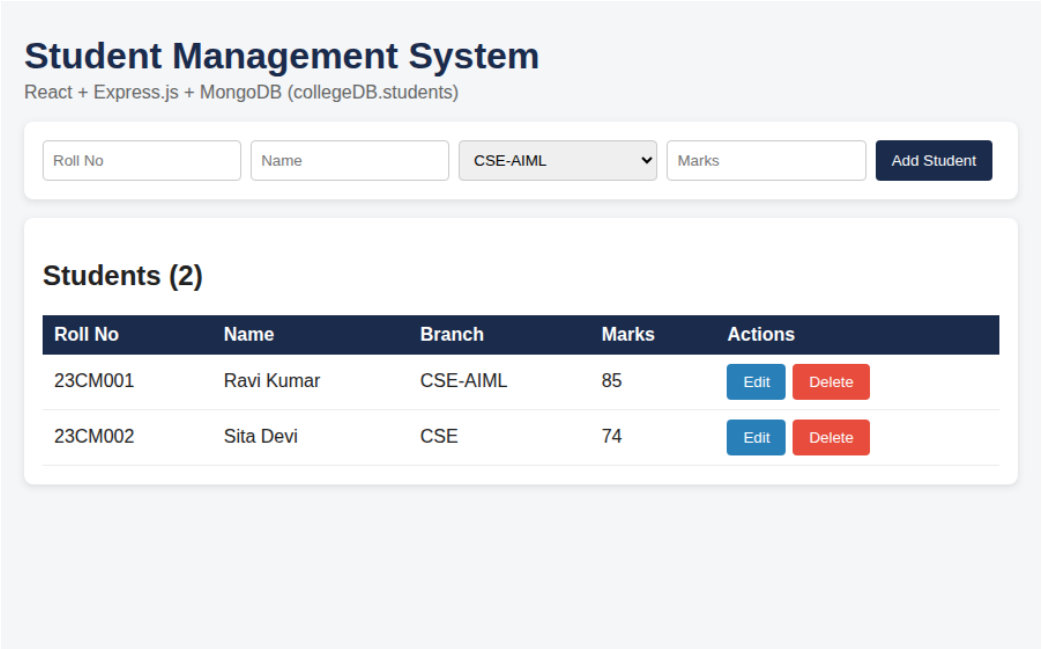
TC-08 — *An error message is shown and nothing is saved*



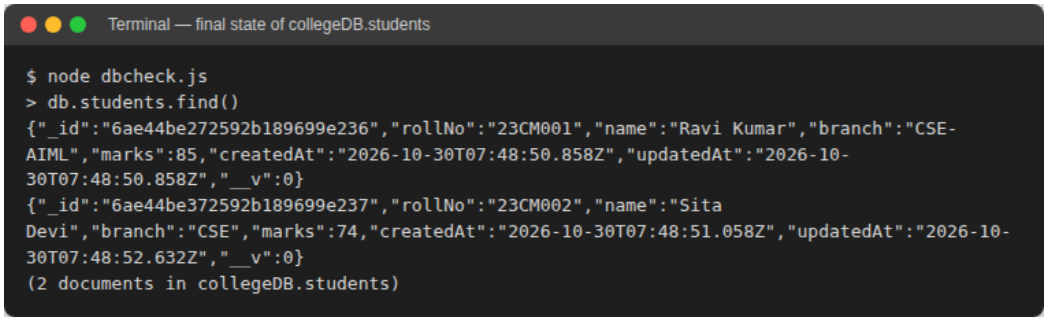
Submitting with the Name empty shows the server's validation message; the list is unchanged

9. OUTPUT

Output: Complete Application and Final Database State



Final view of the application in the browser after all operations



Final contents of collegeDB.students (Sita Devi's marks are 74; Arjun Reddy is deleted)

10. RESULT

Thus, a full-stack Student Management application was successfully developed using ReactJS, Express.js and MongoDB. Adding, viewing, editing and deleting students worked end to end, the data persisted in the database across page reloads, and invalid input was rejected with a validation message.